Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	14 October 2026
Team ID	
Project Name	Metro Ticket Generating System
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Technical Architecture: Metro Ticket Generating System (built on ServiceNow)

The solution is a 4-layer cloud application: Users → Service Portal (presentation) → ServiceNow platform (application logic) → ServiceNow database (data).

Technical Architecture – Metro Ticket Generating System (ServiceNow)

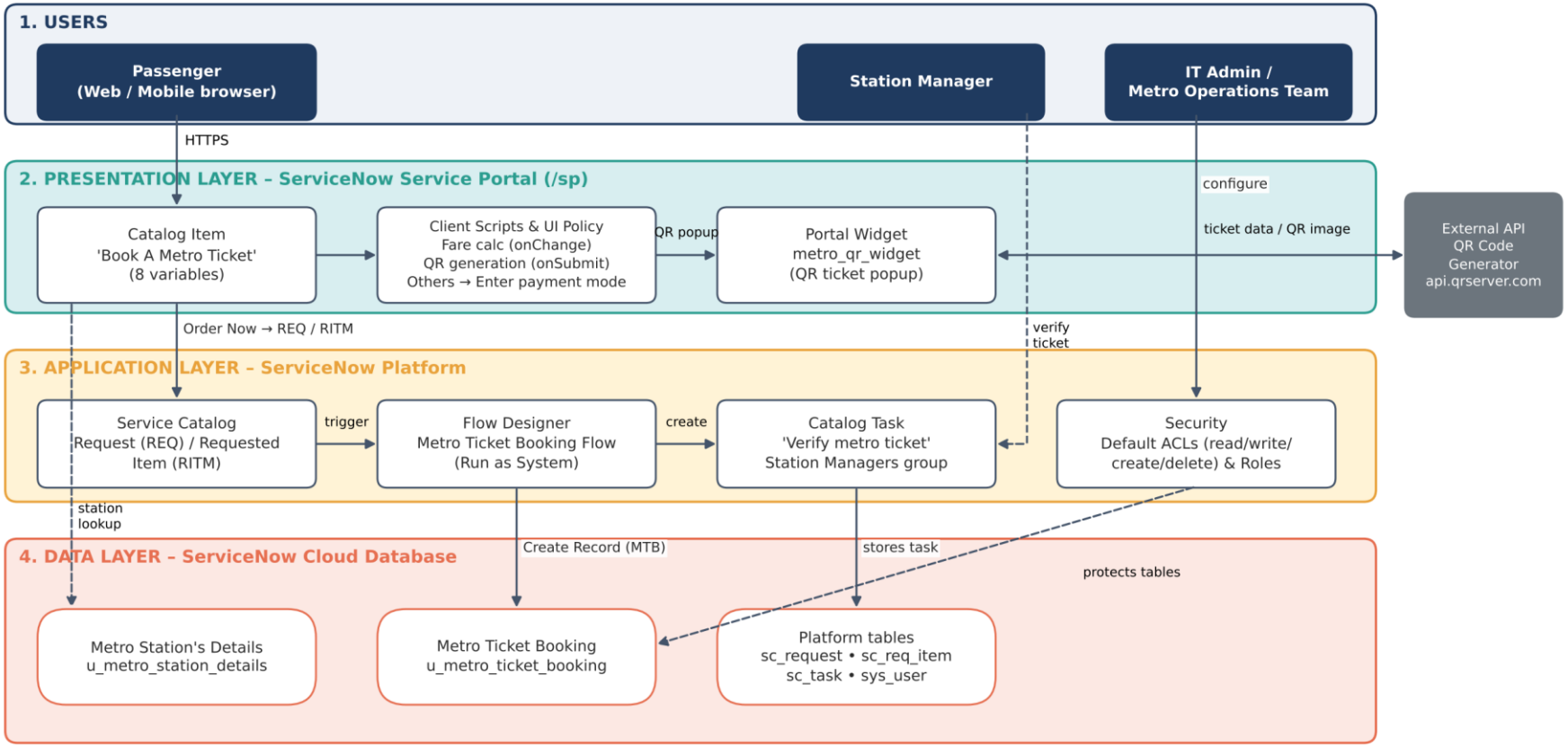


Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	Passengers book tickets through a web form on the ServiceNow Service Portal (/sp) in a desktop or mobile browser; QR ticket shown in a popup	ServiceNow Service Portal, Service Catalog item, HTML / AngularJS widget (metro_qr_widget)
2.	Application Logic-1	Automatic fare calculation for the selected route, journey type and number of passengers	Catalog Client Script (JavaScript, onChange, g_form API)
3.	Application Logic-2	QR ticket generation on Order Now; second click places the order; UI policy shows 'Enter payment mode' for Others	Catalog Client Script (onSubmit), spModal, Catalog UI Policy
4.	Application Logic-3	Store each booking and create a verification task for the Station Managers group	ServiceNow Flow Designer (Service Catalog trigger, Get Catalog Variables, Create Record, Create Catalog Task)
5.	Database	Custom tables: Metro Station's Details (station names) and Metro Ticket Booking (MTB records: stations, journey, passengers, amount, payment, status)	ServiceNow tables (u_metro_station_details, u_metro_ticket_booking)
6.	Cloud Database	Platform tables for requests, requested items, tasks and users	ServiceNow cloud database (sc_request, sc_req_item, sc_task, sys_user)
7.	File Storage	No separate file storage needed; QR images are generated on demand and not stored	Not required (ServiceNow attachments available if needed)
8.	External API-1	Generate the QR code image from the ticket data	QR Code Generator API (api.qrserver.com, HTTPS GET)

9.	External API-2	Payment processing (future scope – UPI / card gateway)	Not used in Phase 1 (e.g. Razorpay / PhonePe in future)
10.	Machine Learning Model	Not used – fares are calculated with fixed rules per route	Not applicable (rule-based logic)
11.	Infrastructure (Server / Cloud)	Application runs on a ServiceNow Personal Developer Instance in the cloud Local Server Configuration: not required – only a web browser Cloud Server Configuration: ServiceNow PDI (Global scope), update set 'Metro Ticket Booking'	ServiceNow Cloud (PDI)

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	AngularJS used inside the Service Portal widget; Bootstrap styling of the portal	AngularJS, Bootstrap (bundled with ServiceNow Service Portal)
2.	Security Implementations	Default ACLs (read, write, create, delete) on custom tables; role-based access; users book only through the catalog; flow runs as System; HTTPS for all traffic	ServiceNow ACLs, Roles, HTTPS / TLS
3.	Scalable Architecture	3-tier design (portal UI, platform logic, database); new stations and routes are added as data without code changes	ServiceNow multi-tenant cloud, Service Catalog, Flow Designer
4.	Availability	Hosted on ServiceNow's cloud data centres, available 24×7 through the internet	ServiceNow Cloud (PDI)
5.	Performance	Fare calculated in the browser (no server call); QR image fetched directly from the API; flow runs asynchronously after submission	Client-side scripts, asynchronous Flow Designer execution

References:

https://c4model.com/

https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/

https://www.ibm.com/cloud/architecture

https://aws.amazon.com/architecture

https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d